



'K' Closest Points to the Origin (easy)

We'll cover the following



- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given an array of points in a $2D$ plane, find 'K' closest points to the origin.

Example 1:

Input: points = [[1,2],[1,3]], K = 1

Output: [[1,2]]

Explanation: The Euclidean distance between (1, 2) and the origin is $\sqrt{5}$.

The Euclidean distance between (1, 3) and the origin is $\sqrt{10}$.

Since $\sqrt{5} < \sqrt{10}$, therefore (1, 2) is closer to the origin.

Example 2:

Input: point = [[1, 3], [3, 4], [2, -1]], K = 2

Output: [[1, 3], [2, -1]]

Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```
1 import java.util.*;
2
3 class Point {
4     int x;
5     int y;
6
7     public Point(int x, int y) {
8         this.x = x;
9         this.y = y;
10    }
11
12    public int distFromOrigin() {
```



```

13     // ignoring sqrt
14     return (x * x) + (y * y);
15 }
16 }
17
18 class KClosestPointsToOrigin {
19
20     public static List<Point> findClosestPoints(Point[] points, int k) {
21         ArrayList<Point> result = new ArrayList<>();
22         // TODO: Write your code here
23         return result;
24     }
25
26     public static void main(String[] args) {
27         Point[] points = new Point[] { new Point(1, 3), new Point(3, 4), new Point(2, -1) };
28         List<Point> result = KClosestPointsToOrigin.findClosestPoints(points, 2);
29         System.out.print("Here are the k points closest the origin: ");
30         for (Point p : result)
31             System.out.print "[" + p.x + " , " + p.y + " ] ";
32     }
33 }
34

```

Run

Save

Reset



Solution

The [Euclidean distance](#) of a point P(x,y) from the origin can be calculated through the following formula:

$$\sqrt{x^2 + y^2}$$

This problem follows the [Top 'K' Numbers](#) pattern. The only difference in this problem is that we need to find the closest point (to the origin) as compared to finding the largest numbers.

Following a similar approach, we can use a **Max Heap** to find 'K' points closest to the origin. While iterating through all points, if a point (say 'P') is closer to the origin than the top point of the max-heap, we will remove that top point from the heap and add 'P' to always keep the closest points in the heap.

Code

Here is what our algorithm will look like:

```

1  import java.util.*;
2
3  class Point {
4      int x;
5      int y;
6
7      public Point(int x, int y) {
8          this.x = x;
9          this.y = y;
10     }
11
12     public int distFromOrigin() {
13         // ignoring sqrt
14         return (x * x) + (y * y);
15     }
16 }

```



```
17
18 class KClosestPointsToOrigin {
19
20     public static List<Point> findClosestPoints(Point[] points, int k) {
21         PriorityQueue<Point> maxHeap = new PriorityQueue<>((p1, p2) -> p2.distFromOrigin() - p1.distFromOrigin());
22         // put first 'k' points in the max heap
23         for (int i = 0; i < k; i++)
24             maxHeap.add(points[i]);
25
26         // go through the remaining points of the input array, if a point is closer to the origin than the top point
27         // of the max-heap, remove the top point from heap and add the point from the input array
28         for (int i = k; i < points.length; i++) {
29             if (points[i].distFromOrigin() < maxHeap.peek().distFromOrigin()) {
30                 maxHeap.poll();
31                 maxHeap.add(points[i]);
32             }
33         }
34
35         // the heap has 'k' points closest to the origin, return them in a list
36         return new ArrayList<>(maxHeap);
37     }
38
39     public static void main(String[] args) {
40         Point[] points = new Point[] { new Point(1, 3), new Point(3, 4), new Point(2, -1) };
41         List<Point> result = KClosestPointsToOrigin.findClosestPoints(points, 2);
42         System.out.print("Here are the k points closest the origin: ");
43         for (Point p : result)
44             System.out.print "[" + p.x + " , " + p.y + " ] ";
45     }
46 }
47
```

Run

Save

Reset



Time complexity

The time complexity of this algorithm is $(N * \log K)$ as we iterating all points and pushing them into the heap.

Space complexity

The space complexity will be $O(K)$ because we need to store 'K' point in the heap.

[< Back](#)☒ Mark As Completed[Next >](#)

Kth Smallest Number (easy)

Connect Ropes (easy)